Date

ILLUSTRATING THE RELATIONSHIP WITH BINARY AND ASSEMBLY LANGUAGE.

Consider a table showing location (memory address of existence) of an instruction in first column and then the instruction itself in the next column.

BINARY PROGRAM TO ADD TWO NUMBERS.

Location.	Instruction Code.
0	0010 0000 0000 0100
1	0001 0000 0000 0101
10	0011 0000 0000 0110
11	0111 000 0000 0001
100	000 0000 0101 0011
101	1111 1111 1110 1001
110	0000 0000 0000 0000

The Binary representation of our program is quite difficult therefore we can represent it in an Hexadecimal equivalent table.

HEXADECIMAL PROGRAM TO ADD TWO NUMBERS.

Location	Instructions.
000	2004
001	1005
002	3006
003	7001
004	0053
005	FFE9
006	0000

Date

Now further we can use symbolic representation of the opcode to make the interpretation of our program much easier. with inclusion of another column to provide insights of each statement.

PROGRAM WITH HEXADECIMAL + SYMBOLIC CODE RERESENTATION.

Location	Instruction	COMMENTS.
000.	LDA 004	Load first operand to AC
001.	ADD 005	Add second operand to AC
002.	STA 006	Store sum in location 006
003	HLT	Halt Computer.
004	0053	First operand.
005	FFE9	second operand (neg ative)
006	0000	Store sum here.

as the leftmost position is 1 i.e negative for a sign bit.

Now we can go one step ahead and replace each hexadecimal address with/by a symbolic address and each hexa-decimal operand by a decimal operand.

Date

equivalent Fotran program is.

```
INTEGER A, B, C          C = A + B
DATA A, 83 B, -23        END.
```

} converted to Binary by compiler

ASSEMBLY LANGUAGE PROGRAM TO ADD NO. :-

```
ORG 0          / Origin of the program at location 0
LDA A          / Load operand from location A
ADD B          / Add operand from location B
STA C          / store sum in location C.
HLT            / Halt Computer.
A,  DEC 83     / Decimal Operand.
B,  DEC -23    / Decimal operand.
C,  DEC 0      / End of Symbolic Program.
```

NOTE

ORG; is not a no. rather also not a machine language instruction Its purpose is to specify an 'origin' i.e the memory location of the next instruction below it.

The next 3 lines have symbolic addresses. Their value is specified by the labels present in the 1st column.

DEC operands are specified by the symbol DEC. The decimal no.s may be positive or negative but negative no.s will be converted to binary by 2's complement representation.

The symbols ORG, DEC and END are called pseudo instructions (4x in our system).

↳ not machine language inst; rather an inst. giving assembler infor about some phase of translation

END: to inform assembler about program termination.

other two including DEC tell the assembler how to convert to Binary (i.e specify radix of the operand). & HEX

*ORG: tells assembler that the instruction operand of the line is to be placed in a memory location (specified by the no. next to ORG. → (following)

Date
ORG can be used more then once to specify various segments of the memory.

ASSEMBLY LANGUAGE:-

The Basic unit of an assembly language program is a line of code.
The specific language is defined by a set of rules that specify the symbols that can be used and how they can be combined to form a line of code.

RULES OF ASSEMBLY LANGUAGE:-

Each line of an assembly language program is arranged in three columns called fields. The fields may specify the following information.

(i) The label field may be empty or it may specify a symbolic address.

(ii) The instruction field specifies a machine instruction or a pseudo-instruction.

(iii) The comment field may be empty or it may include a comment.

(a) LABEL FIELD

Symbolic Address may consist of 1, 2 or 3 alphanumeric characters but not more than that. The first character must be a letter, the next two may be letters or numerals
The symbolic address field is chosen arbitrarily

Date

and must be terminated by a comma; so that the assembler recognizes it as a label.

INSTRUCTION FIELD:-

Instruction may specify one of the following.

(1) A memory reference instruction (MRI)
(2) A register-reference or input/output instruction (Non-MRI).
(3) A pseudo instruction with or without operand.

(compulsory!) 1st operation + symbolic Address
↗ ↗ optional as situation

A memory RI may occupy two or three symbols separated by spaces. The first (3 letter) symbol defines MRI operation; second is a symbolic Address and third is the letter (I)

if

Absent — direct add inst
Present — indirect address inst.

EXAMPLE

CLA — Non MRI
ADD OPR — Direct MRI.
ADD PTR I — indirect MRI.

symbolic address as a label

* this should be defined somewhere in memory as a label appearing in the first column

Date

Question: Assembly language Program to substract two No.s.

```
        ORG 100      ------>  / origin of the program is location100
        LDA SUB      ------>  / load subtrahend to AC
        CMA          ------>  / Complement AC
        INC          ------>  / Increment AC
        ADD MIN      ------>  / Add minuend to AC
        STA DIF      ------>  / store difference.
        HLT          ------>  / Halt computer.
MIN,    DEC 83       ------>  / minued
SUB,    DEC -23      ------>  / Subtrahend.
DIF,    HEX 0        ------>  / store difference here.
        END          ------>  end symbolic program.
```

(Hex code can easily be interpreted in Binary).

CONVERSION TO BINARY

100	2107	ORG 100 LDA SUB
101	7200	CMA
102	7020	INC
103	1106	ADD MIN
104	3108	STA DIF
105	7001	HLT
106	0053	DEC 83
107	FFE9	DEC -23
108	0000	HEX 0

END.

* process of conversion of a symbolic program to Binary is done by means of an assembler.

* ORG & END are not assigned a numerical location because they do not represent an instruction or operand.

Address symbol	Hexadecimal Address
MIN	106
SUB	107
DIF	108

NOTED

This process of conversion takes place in two primary steps.

(i) The translation of symbols to Binary; especially in the first step we assign memory location to each machine instruction and operand sequentially. called as the first pass

(ii) During the second pass of the symbolic program we refer to the address symbol table to determine the address value of memory-reference inst. and Binary translation occurs.
e.g LDA SUB is translated into 2107.

'ASSEMBLER'

An assembler is a program that accepts a symbolic language program and produces its Binary machine language; equivilent.

Input : source program.

output : object program.

* Assembler operates in character strings to produce an equivalent Binary program.

of symbolic program.

⇒ Before the assembly process the symbolic program is to be stored in memory using a loader program which for each character of the symbolic program converts it into an 8 Bit Binary code (this as an equivalent hexadecimal code too)

The highest order Bit here is 0 and the remaining 7 are decided by the ASCII code. (Table 6-10).

Date

symbolic program in memory.

Example: Representing ↑ a line of code
PL3, LDA SUBI

NOTE:

* In table 6-10 we saw that each character of a symbolic program is equivalent to 2 hexadecimal digits

* as we know that our memory word is only 16 Bit, we can thus use only two Symbols of the ↓ memory location
symbolic program in each

Remember:

* label is terminated by comma.
operation & Address symbol is terminated by space

Solution: following Rules

↑ from table 6-10 → convert to Binary →

1	P	L	50 4C	0101 0000 0100
2	3,		33 2C	0011 0011 0010
3	L	D	4C 44	0100 1100 0100
4	A		41 20	0100 0001 0010
5	S	U	53 55	0101 0011 0101
6	B		42 20	0100 0010 0010
7	I	CR	49 0D	0100 1001 0000

similarly also convert (C, C, 4, 0, 5, 9D)

Date

NOTE :-

The input of the Assembler is then the user's symbolic language program in ASCII.
The input is then scanned by the assembler twice to produce an equivalent Binary program. (output).

WORKING OF AN ASSEMBLER.

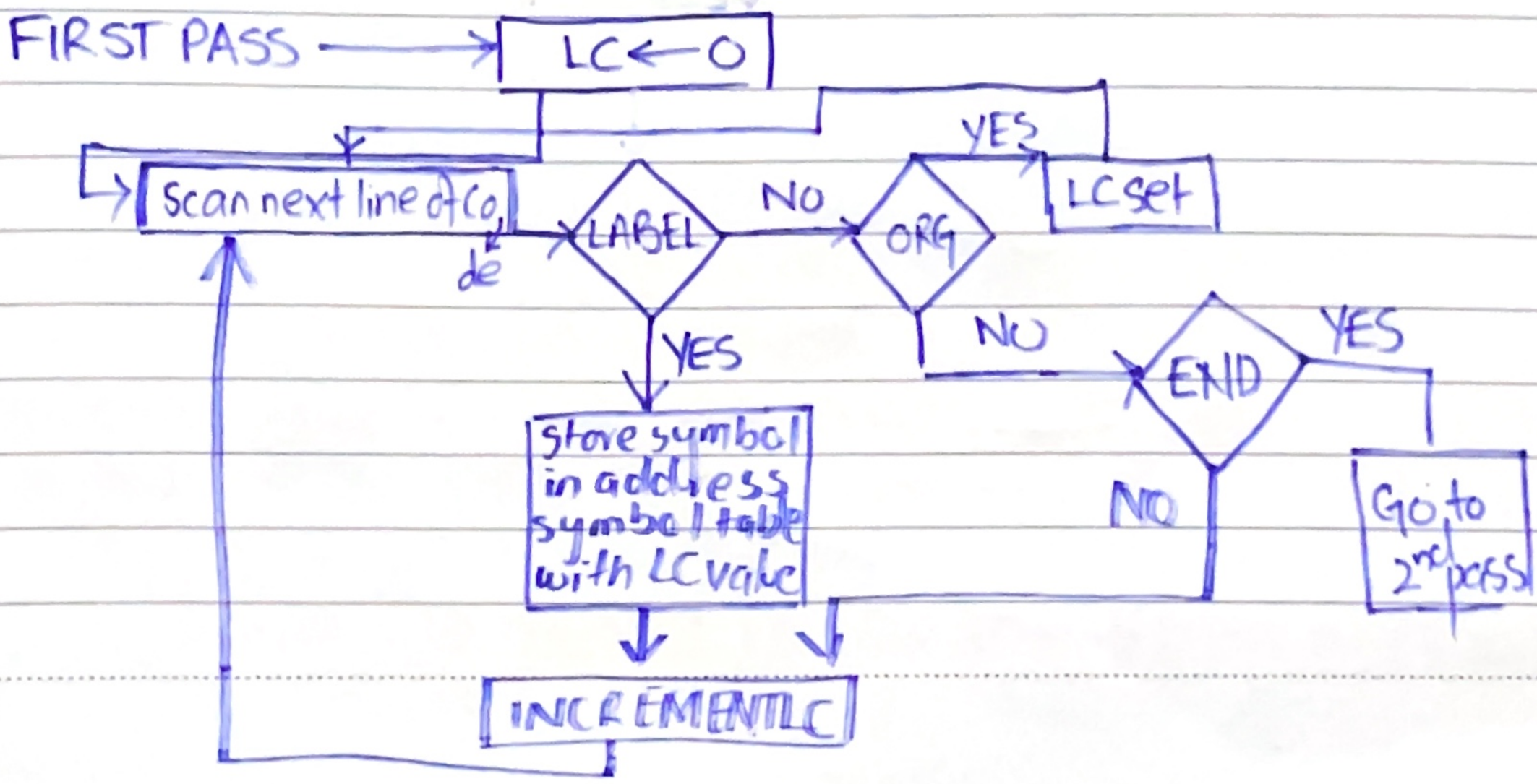
'FIRST-PASS'

Two pass assembler converts the entire symoblic code into Binary in two passes.

① The first pass generates a table that corelates all user defined address symbols with their Binary equivalent values.
② The Binary translation is done during the second pass.

LOCATION COUNTER :-

LC keeps track of the location of instructions in an assembler, its value is +1 éd to access the next value sequentially.
If ORG does not define the begining of instruction location, by default the assembler sets the LC to 0 initally.



Date

address symbol

Example: Previously we have learned the assembly code for substracting two numbers now letus find the table generated for it during the first pass (for labels) by the assembler.

Memory Word	Symbol & (LC)*	Hexadecimal code.	Binary Representation
1	M I	4D 49	
2	N ,	4E 2C	
3	(LC)	01 06	
4	S U	53 55	
5	B ,	42 2C	
6	(LC)	01 07	
7	D I	44 49	
8	F ,	46 2C	
9	(LC)	01 08.	

*(LC) designates the content of Location Counter.

Date

WORKING OF ASSEMBLER:-

"SECOND PASS"

↳ Machine instructions are translated here by means of a table lookup procedure; which is a search of table entries to determine whether a specific item matches one of the items stored in the table.
If not then the symbol is not interpreted.

We have the following four tables.

(i) Pseudo-Instruction table. ⟶ END, ORG, DEC, HEX
(ii) MRI Table.
(iii) Non-MRI Table.
(iv) Address symbol Table.

↓ each refers the assembler to a sub-routine that processes the pseudo-instructions when encountered in the program.

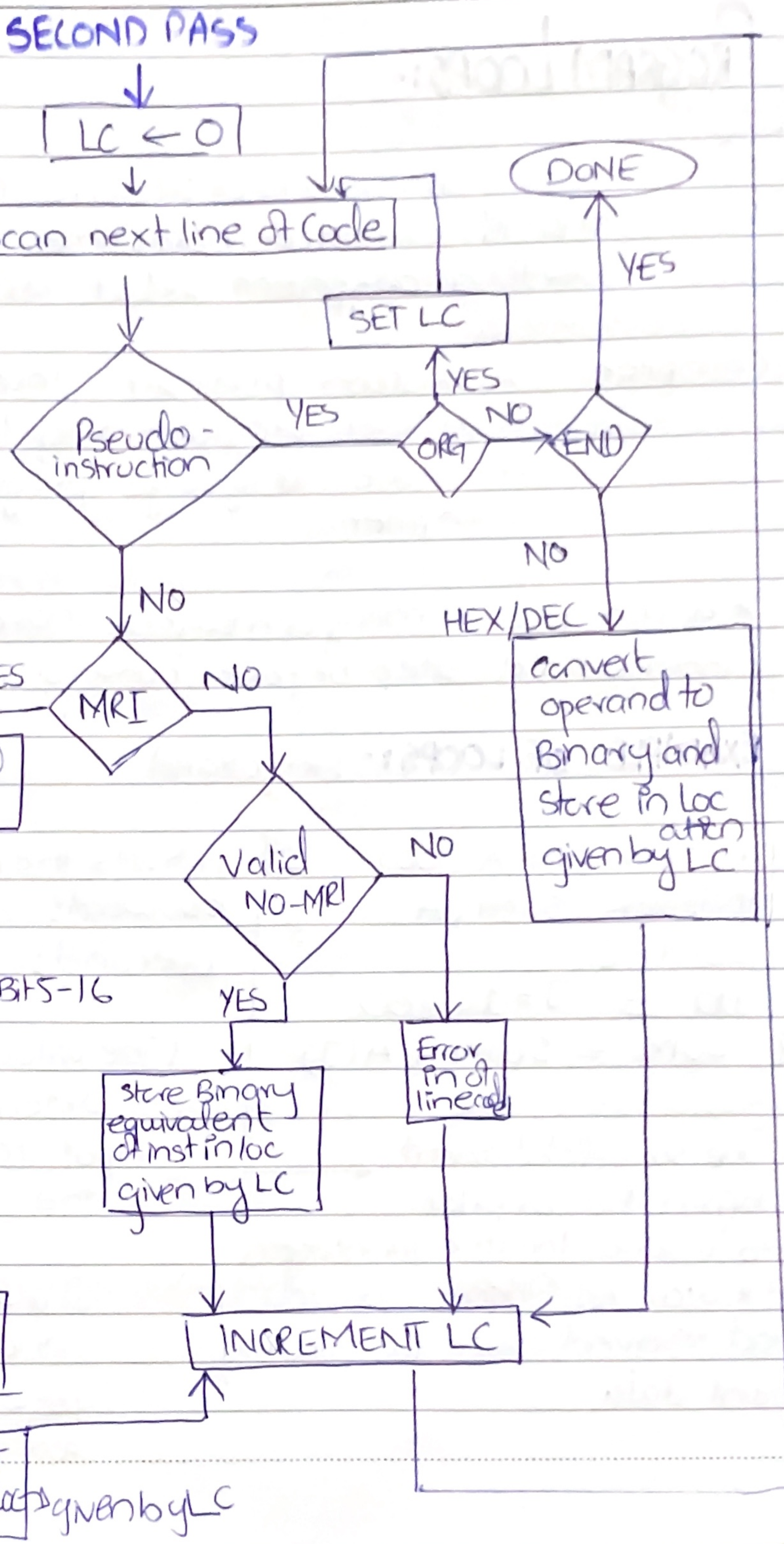
↓ If this is found, the assembler stores the 16-Bit instruction in the memory word and LC=LC+1

* at this's the assembler extracts the 3-Bit Opcode to analyze the next instruction. and puts it in 2 through 4 of a word.

* ERROR DIAGNOSTICS; (error occurs when)
(i) The assembler cannot translate a symbol because it doesnot know its Binary equivalent.
(ii) having a symbolic address that doesnot occur as a label.

Date

SECOND PASS



Date

PROGRAM LOOPS:-

A program loop is a sequence of instruction that is executed many times, each time, with a different set of data.

Compiler: A system program that converts the high level programming language into a machine language program is called a compiler.

* a compiler may or may not have assembly language as an immediate step before conversion to Binary.

EXAMPLE OF LOOPS:- (in fotran)

```
DIMENSION A(100)        ①
INTEGER SUM, A          ②
SUM = 0
DO 3 J = 1, 100
3 SUM = SUM + A(J)
```

① → instructs the compiler to reserve 100 words of memory for 100 operands.

(The values of the operands is determined by the input statement, not listed in the program).

The second statement informs the compiler to reserve location for integer; if it was real then it would had reserved space for floating point data

① & ② are non executable and similar to pseudoinstruction is assembly language.

line nos are for reference only.

SYMBOLIC PROGRAM TO ADD 100 NUMBERS.

```
1                ORG 100          / origin of the program is HEX 100
2                LDA ADS          / load address of first operand to accumulator.
3                STA PTR          / store in pointer.
4                LDA NBR          / load minus 100
5                STA CTR          / store in counter.
6                CLA              / Clear accumulator.
7   LOP,         ADD PTR I        / add operand to AC.   (I = indirect)
8                ISZ PTR          / Increment pointer.
9                ISZ CTR          / Increment Counter.
10               BUN LOP          / Repeat loop
11               STA SUM          / store sum.
12               HLT
13  ADS,         HEX 150          / first address of operands.
14  PTR,         HEX 0            / Location is reserved for ptr.
15  NBR,         DEC -100         / Constant to initialize ctr.
16  CTR,         HEX 0            / Location reserved for counter.
17  SUM,         HEX 0            / store SUM here.
18               ORG 150
19               DEC 75
.
.
.
               DEC 23
118              END.
```

* Read very important explanation from page 200.

Date

PROGRAMMING ARTHEMATICS & LOGIC OPERATIONS.

Operations that are implemented on a computer with one machine instruction are said to be implemented by hardware.
Operations implemented by a set of instructions that constitues a program are said to be executed / implemented by software.

* Hardware implementation is more costly because of the additional circuits needed to be implemented for few arthematic and logic operations.

* Software implementation results in long programs both in numbers of inst. and execution time.

MULTIPLICATION PROGRAM:-

Pen & Pencil method.

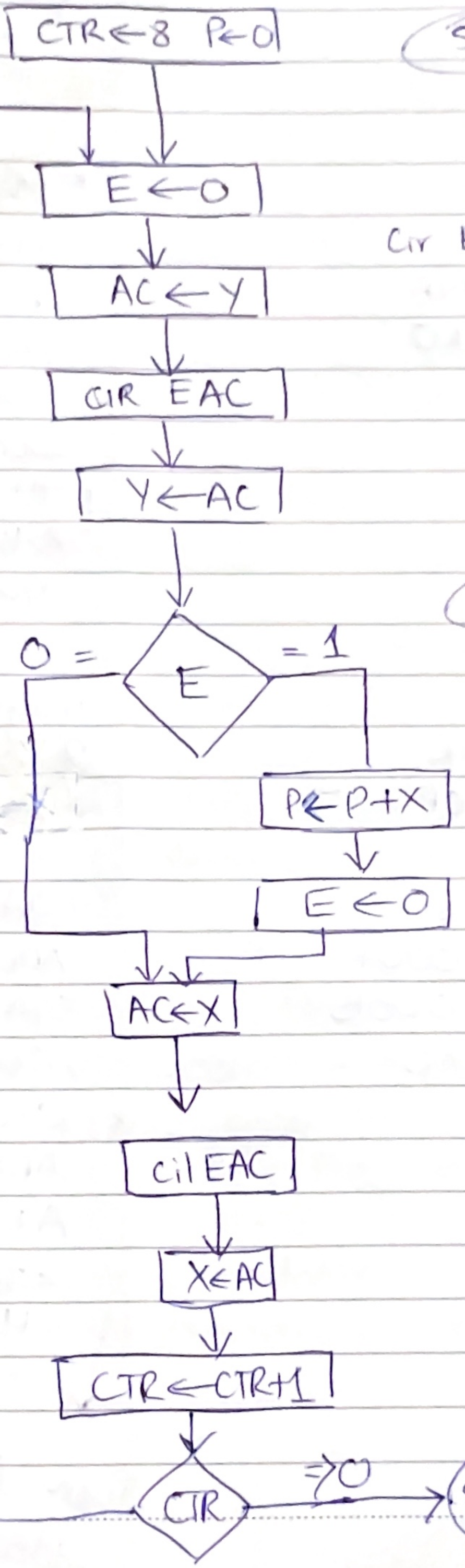
Consider the following example. (14 x10)

multiplicand ↑ , ↳ multiplier.

1110	(14)	00001110.	00001010
1010.	(10)		
0000		Partial Product	
1110X		Partial Product	
0000XX		Partial Product	
1110XXX		Partial Product.	
10001100			

By default to keep things simple we have considered 8Bits only ∴ 8×8 ⇒ max can result in 16 Bits answer.

Date



step no. 1

Y = 00001010 (initial)

AC = "

Cir EAC = 000001010

E = 0; Y = 00000101

X = 00001110 (initial)

AC = "

Cir EAC = 00011100

X = 00011100

step no. 2

Y = 00000101

AC = "

Cir EAC = 10000010

E = 1 Y = "

X = 00011100

AC = "

X = 00111000

P = 08111100

step no. 3.

Y = 10000010

AC = "

Cir EAC = 01000001

E = 0 Y = "

X = 00111000

Cir EAC = X ⇒

01110000

X = "

step no. 4

X = 01110000

Y = 01000001

Cir EAL(Y) = 10100000

E = 1

P = P+X

00111100

⇒ 01110000

10001100

Ans.

X = 11100000

Date (ASSEMBLY LANGUAGE CODE)

```
      ORG 100.
LOP,  CLE
      LDA Y
      CIR
      STA Y
      SZE
      BUN ONE
      BUN ZRO
ONE   LDA X
      ADD P
      STA P
      CLE
ZRO   LDA X
      CIL
      STA X
      ISZ CTR
      BUN LOP
      HLT
CTR   DEC -8
 X,   HEX 000F
 Y,   HEX 000B
 P,   HEX 0
      END.
```

DOUBLE PRECISION ADDITION

* Multiplying 2 16Bit unsigned nos will results in 32Bit product that must be stored in 2 memory words.

→ the no. is said to have double precision

Program to add 2 Double Precision Numbers.

```
LDA  AL
ADD  BL
STA  CL
CLA
CIL
ADD  AH
ADD  BH
STA  CH
HLT.
```

* CIL → Brings carry from lower order addition to 0th position of AC.

then AH is added with carry

then sum added with BH.

ADD => Op1 + Op2 => (Op1' * op2')'

Date

LOGICAL SHIFT OPS

① Logical shift Right. CLE & CIR.

② Logical shift left. CLE & CIL

ARTHEMATIC RIGHT SHFT:-

```
CLE     // Clear E to 0
SPA     // Skip if AC is positive.
CME         E remains 0.
CIR.
```

↘ we assign E the same Bit as the sign Bit (leftmost Bit) and perform Arthematic right shift.

~ x ~

The process of branching to a subroutine and returning to a main program is commonly known as subroutine linkage.

BSA inst perform the op ⟶ called as subroutine call.
last subrotine inst is called as ⟶ subroutine return.

↗ (also know as parameters)
Transferring data to & fro from the subroutines is done in two ways

(i) Storing data in Accumulator.
(ii) Accessing through memory location.

day / date:

COAL

Chapter no.6 CNTD.

SUB-ROUTINES :-

set of common insts that can be used in the code several times.

Each time that sub-routine is used in the main part of the program, a branch is executed to the beginning of that subroutine; after the subroutine is executed the branch is made back to the main program.

+ subroutine stores / knows the return address.

EXAMPLE : SUBROUTINE TO SHIFT THE CONTENTS OF ACCUMULATOR 4X.

```
        ORG   100
100     LDA X
101     BSA SH4
102     STA X
103     LDA Y
104     BSA SH4 // Branch to subroutine
105     STA Y
106     HLT
107  X, HEX 1234
108  Y, HEX 4321
109  SH4, HEX 0 // stores the return add here.
```

```
        CIL
        CIL
        CIL
        CIL
        AND MASK
        BUN SH4 I
MSK, HEX FFF0
        END
```

day / date:

The computation in the subroutine circulates the contents of AC 4 times to the left. In order to accomplish a logical shift operation, the four low-order bits must be set to 0. This is done by masking FFF0 with the contents of AC.

* A Mask operation is an AND operation that clears the bits of AC where the mask operand is zero and leaves the bits of AC unchanged where the mask operand is 1.

NOTE:-

The process of branching to a sub-routine and returning to the main program is called as the subroutine linkage.

* we see the first memory location of each sub-routine serves as a link between the main program and the subroutine.

* BSA inst. performs the operation commonly called as the subroutine call.

* The last inst of the subroutine performs an operation called as the sub-routine return

day / date:

CONCEPT OF INDEX REGISTERS:-

The procedure of sub-routine linkages is usually found in computers with 1 processor register. Many computers have multiple processor registers and some of them are assigned the name "Index registers".

↳ usually employeed to implement the subroutine linkage

A Branch-to-subroutine linkage inst stores the return address in an index register. A return from subroutine inst is effected by branching to the address presently stored in the index register.

SUBROUTINE PARAMETERS & DATA LINKAGES:-

In the prev e.g., the accumulator was already loaded with data for the sub-routine to work with; however in general the subroutine should have access to data to work with and return the output to the (explicit) main program.

This can be achieved by

(i) Using an accumulator for a single input parameter and a single output parameter
↳ as done previously.

Data

(ii) Transfer through Memory ...> P.T.O

day / date:

Data are often placed in memory locations following the call. They can also be placed in a block of storage.
The first address of the block is then placed in the memory location following the call

EXAMPLE: PROGRAM TO DEMONSTRATE PARAMETER LINKAGE.

```
        ORG   200.
200     LDA X          / load the 1st op to AC.
        BSA OR         / Branch to subroutine
        HEX 3AF6       / 2nd op stored here.
        STA Y          / subroutine returns here
        HLT
X,      HEX 7B95       / 1st op store here.
Y,      HEX 0          / Result stored here.
OR,     HEX 0          / subroutine start
        CMA            / complement 1st op.
        STA TEMP       / Store in temp loc.
        LDA OR I       / Load second op.
        CMA            / complement 2nd op.
        AND TMP        / → self explanatory
        CMA            /   "
        ISZ OR         / Increment return add.
        BUN OR I       / Return to main prog.
TEMP,   HEX 0          / Temp storage.
        END.
```

day / date:

EXAMPLE: SUBROUTINE TO MOVE A BLOCK OF DATA.

A subroutine that moves the block of data starting at address 100 into a block of data starting with address 200.

The length of the block is 16 words.

```
                    / Main program
    BSA MVE         / Branch to subroutine
    HEX 100         / First address of source data.
    HEX 200         / First address of destination data.
    DEC -16         / Number of items to move.
    HLT
MVE, HEX 0
    LDA MVE I
    STA PT1
    ISZ MVE
    LDA MVE I
    STA PT2
    ISZ MVE
    LDA MVE I
    STA CTR
    ISZ MVE
LOP, LDA PT1 I
    STA PT2 I
    ISZ PT1
    ISZ PT2
    ISZ CTR
    BUN LOP
    BUN MVE I
```

PT1, ____ | CTR, ____

PT2, ____

day / date:

Input - Output Programming.

Table 6-19:-

(a) Input Character:-

Program to Input One character.

```
CIF,    SKI     / Check input flag to see if the char
                  is available for transfer.
        BUN CIF / Flg = 0, branch to check again
        INP     / Flag = 1, Input character.
        OUT     / Print character.
        STA CHR / store character.
        HLT.
CHR,
```

(b) Output Character:-

```
        LDA CHR     / Load character into AC.
COF,    SKO         / Check the output flag.
        BUN COF     / Flag=0, branch to check agg
        OUT         / Flag = 1 Output character.
        HLT
CHR,    HEX 0057    /character is 'W'
```

Concept of character manipulation:-

Binary coded characters that represent symbols can be manipulated by computer instruction to achieve various data-processing tasks.